GITHUB WALKTHROUGH

GlowUp Repository Walkthrough

A structured overview of the Flutter app, its navigation flow, services, data model, and the main user journey from onboarding to workouts, reports, and settings.

PROJECT TYPE
Flutter mobile app

PRIMARY STACK
Firebase, SharedPreferences, TTS, ads, camera tracking

MAIN GOAL
Guided 30-day glow-up challenge

1. Repository Overview

GlowUp is a Flutter app built around a structured 30-day challenge. The app guides the user through onboarding, optional authentication, fitness-level personalization, daily workouts, progress tracking, and a settings area for account and app preferences.

The project is organized as a standard Flutter monorepo with Android, iOS, web, Linux, macOS, and Windows folders, while the primary product logic lives under `lib/`.

What this app does

It runs a day-by-day challenge, adapts reps by fitness level, stores progress

What makes it notable

It combines a polished consumer UI with fitness logic, Firebase auth, ad

locally, and uses voice guidance plus camera-based tracking during workouts.

support, voice coaching, and data persistence in a single mobile workflow.

2. App Architecture

The entrypoint in `lib/main.dart` initializes Firebase, local data, and ads before deciding where the user should land. The routing logic checks onboarding, authentication, and personalization flags stored in SharedPreferences and then sends the user to the next unfinished step.

Layer	Purpose
Presentation	Flutter screens, widgets, animations, and theme handling
State and persistence	SharedPreferences for onboarding, auth, current day, streaks, and fitness level
Workout logic	Exercise plans, rep counts, rest timers, completion flow, and voice cues
External services	Firebase Auth, Google Sign-In, ads, TTS, and camera tracking

3. End-to-End User Flow

1. The app boots in `main.dart`, initializes services, and chooses the next screen.
2. If onboarding is not complete, the user sees the three-page onboarding sequence.
3. If auth is required, the user goes to the authentication screen and can sign up, sign in, use Google, or skip.
4. The user chooses a fitness level on the personalization screen.
5. The main shell opens with three tabs: Report, Home, and Settings.
6. The user starts the day’s workout from Home, completes exercises with rep or timer tracking, and lands on the completion screen.
7. Progress is reflected in Report and can be managed from Settings.

The app is designed as a funnel: onboarding -> auth -> personalization -> challenge delivery -> progress retention.

4. Core Screens

Onboarding

The onboarding screen presents three motivational pages about consistency, healthy eating, and progress tracking. It uses Lottie animations and a single action button that advances page by page before completing onboarding in SharedPreferences.

Authentication

The auth screen supports email/password login and Google Sign-In. It also allows skipping auth, which is persisted so the app does not keep prompting the user once they opt out.

Personalization

The personalization screen asks the user to pick Beginner, Medium, or Advanced. That choice changes the reps assigned across exercises and is saved locally as part of the onboarding journey.

Main Shell

The main shell uses an IndexedStack with three tabs: Report, Home, and Settings. A banner ad is placed below the bottom navigation bar so it remains visible across tabs.

Home

The Home screen is the dashboard. It surfaces the current day, streaks, progress percent, calendar status, and a featured workout card. The layout is intentionally layered and visual-heavy to make the daily challenge feel premium.

Workout

The workout screen is the most logic-heavy part of the app. It loads exercises for the current day, handles rest intervals, plays audio guidance, manages countdowns, and reacts to tracking snapshots. Timer-based exercises and AI-tracked exercises are handled differently.

Workout Complete

After a workout, the app updates streaks and progress, shows a celebratory screen, and triggers an interstitial ad. The user can return to Home from there.

Report and Settings

The Report tab summarizes completed workouts, exercises, minutes, history, streaks, and completed days. Settings covers theme switching, account state, sharing, rating, feedback, privacy policy, and a delete-all-data action.

5. Services and State

File	Role
`lib/services/data_service.dart`	Loads the JSON workout plan, tracks onboarding/auth/personalization flags, current day, streaks, and fitness level
`lib/services/auth_service.dart`	Wraps Firebase Auth and Google Sign-In, including sign up, sign in, sign out, and delete-account support
`lib/services/voice_coach_service.dart`	Uses text-to-speech to guide exercises, countdowns, rest periods, and motivation tips
`lib/services/exercise_rep_counter.dart`	Tracks exercise form and reps using face and pose landmarks, plus hold-based logic for timer exercises
`lib/services/ad_service.dart`	Initializes and shows ads, including banner and interstitial placements

The app relies on SharedPreferences for lightweight persistence. That means the app can restore progress and setup state without a custom backend for core challenge tracking.

6. Data Model and Assets

Workout definitions are stored in `assets/data/exercise_plan.json`. The JSON is loaded by the data service and converted into two main models:

- `Exercise`: id, name, description, duration, reps, image, category, difficulty, and voice instruction.
- `DayPlan`: day number, phase, face exercise ids, body exercise ids, and rest duration.

Assets also include three onboarding animations, a logo, and a hero image used in the home screen. The app is already configured to bundle those assets through `pubspec.yaml`.

Key assets

- `assets/images/logo.jpeg`
- `assets/images/glowup-humans.png`
- `assets/animation/onboarding1.lottie`
- `assets/animation/onboarding2.lottie`
- `assets/animation/onboarding3.lottie`

Core packages

- Firebase Core and Firebase Auth
- Google Sign-In
- Google Mobile Ads
- Flutter TTS
- Camera, Lottie, Share Plus, Url Launcher, Shared Preferences

7. Monetization and Integrations

The app already includes ad support, which suggests a freemium or ad-supported consumer model. Banner ads appear in the main shell and completion screens, and an interstitial ad is shown after a workout is finished.

Firebase Auth enables account creation and sign-in, while the delete-account flow is implemented for compliance. This matters if the app is published to the Play Store and needs to satisfy account deletion requirements.

8. Setup and Notes

The project looks ready to run as a standard Flutter app once dependencies are installed and Firebase is configured. The Android app module already includes a `google-services.json` file, and the repository contains platform folders for the other supported targets.

If you are presenting this project, the strongest summary is: GlowUp is a guided fitness and self-improvement challenge app with onboarding, personalization, workout tracking, local progress persistence, voice coaching, Firebase auth, and ad monetization.

Suggested next step: add screenshots, architecture diagram, and a short demo script if this PDF will be used for investors, recruiters, or product walkthroughs.

Generated from the GlowUp workspace structure and source files under lib/, assets/, and pubspec.yaml.